

Programación III

Desarrollo backend con Spring Boot

UNIDAD 1: Programación Funcional

Clase N° 3

API Stream y Optionals

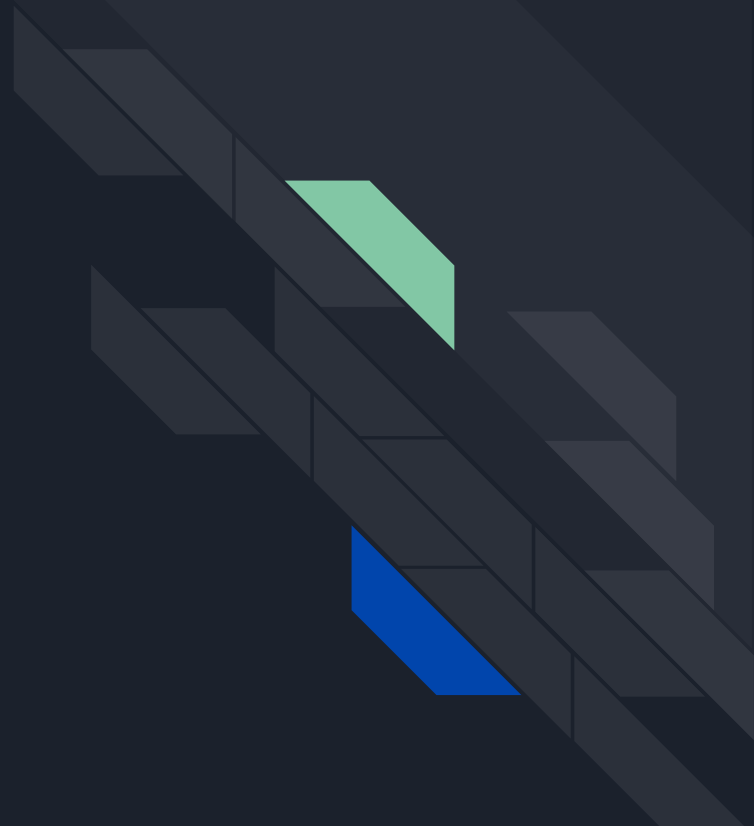
Profesor:

Daniel Díaz



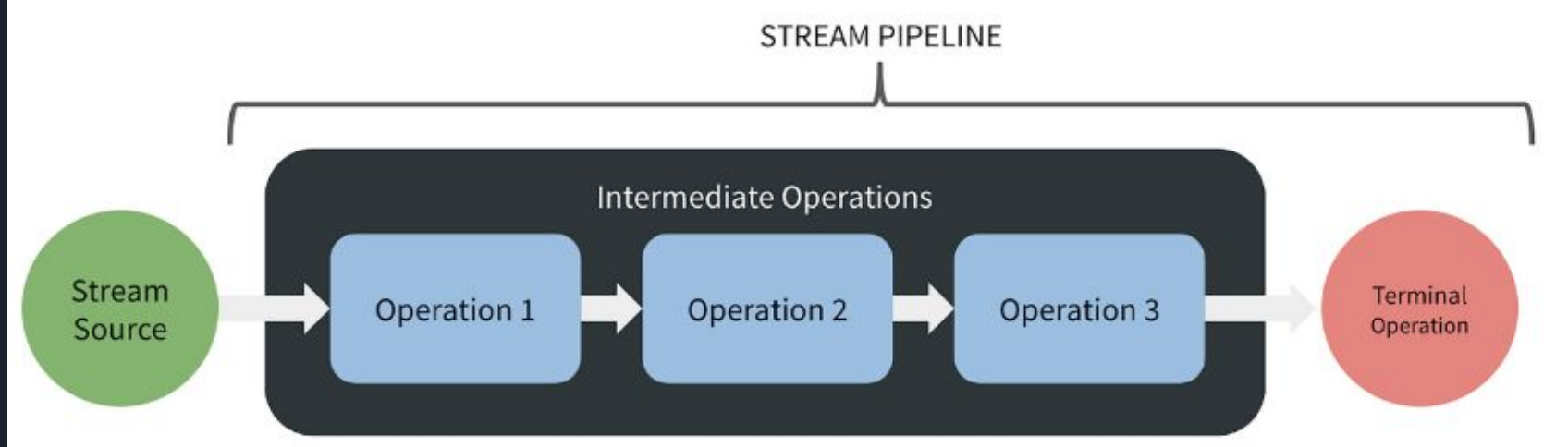
TEMAS A DESARROLLAR

- ¿Qué es un Stream? Diferencias con colecciones.
- Operaciones intermedias vs. terminales.
- Filtrado de elementos con `filter()`.
- Transformación de datos con `map()`.
- Reducción y agregación con `reduce()`.
- Comparación entre for tradicional y Stream API.
- ¿Cuándo usar `parallelStream()`?
- Problema del `NullPointerException` y cómo evitarlo.
- Introducción a `Optional<T>`.
- Métodos útiles de `Optional`



¿Que es un Stream?

Un Stream en Java es una secuencia de elementos (**flujo de datos**) que permite realizar operaciones funcionales como filtrado, mapeo, reducción y agregación sobre colecciones de datos de manera **declarativa y eficiente**, sin modificar la fuente original.



.filter()

El método `filter()` en Java se usa en Streams para filtrar elementos de una secuencia según una condición booleana. Devuelve un nuevo Stream con los elementos que cumplen la condición especificada.

```
public class FilterExample {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
  
        // Filtrar números pares usando toList()  
        List<Integer> numerosPares = numeros.stream()  
            .filter(n -> n % 2 == 0) // Condición: solo pares  
            .toList(); // Método disponible desde Java 16  
  
        System.out.println("Números pares: " + numerosPares);  
    }  
}
```

.map()

El método `map()` en Streams transforma cada elemento de la secuencia aplicando una función y devuelve un nuevo Stream con los elementos modificados. Se usa para realizar mapeos o conversiones sobre los datos.

```
public class MapExample {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);  
  
        // Elevar cada número al cuadrado  
        List<Integer> cuadrados = numeros.stream()  
            .map(n -> n * n) // Transformación: elevar al cuadrado  
            .toList(); // Disponible desde Java 16  
  
        System.out.println("Cuadrados: " + cuadrados);  
    }  
}
```

.reduce()

El método `reduce()` en Streams se usa para reducir una secuencia de elementos a un único valor aplicando una función acumulativa. Es útil para operaciones como sumatoria, producto, concatenación, etc.

```
public class ReduceExample {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);  
  
        // Sumar todos los números de la lista  
        int suma = numeros.stream()  
            .reduce(0, (a, b) -> a + b); // Acumulador: suma de elementos  
  
        System.out.println("Suma total: " + suma);  
    }  
}
```

For vs Streams

Aspecto	<code>for</code> tradicional	Stream API
Paradigma	Imperativo (se indica cómo iterar y modificar datos)	Declarativo (se indica qué hacer, no cómo hacerlo)
Legibilidad	Más verboso	Más conciso y expresivo
Paralelización	No tiene soporte nativo para ejecución paralela	Soporta <code>parallelStream()</code> para procesamiento en paralelo
Eficiencia	Puede ser más eficiente en algunos casos debido a menos sobrecarga	Optimizado internamente, pero con ligera sobrecarga inicial
Inmutabilidad	Generalmente modifica estructuras mutables	Promueve inmutabilidad y evita efectos secundarios

.parallelStream()

El método `parallelStream()` permite procesar los elementos de un `Stream` en paralelo, dividiendo la carga de trabajo en **múltiples hilos** para mejorar el rendimiento en operaciones sobre grandes volúmenes de datos.

```
public class ParallelStreamExample {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
  
        // Sumar los números en paralelo  
        int suma = numeros.parallelStream()  
            .reduce(0, Integer::sum);  
  
        System.out.println("Suma total: " + suma);  
    }  
}
```

¿Cuándo utilizar `parallelStream()`?

✓ Casos recomendados:

- Cuando trabajas con **grandes cantidades de datos** y el procesamiento de cada elemento es costoso.
- Cuando la operación es **independiente** para cada elemento (sin dependencia entre iteraciones).
- Cuando el hardware tiene múltiples núcleos (CPU multinúcleo).

✗ Casos donde NO es recomendable:

- Si la cantidad de datos es pequeña (el costo de gestionar los hilos supera el beneficio).
- Si hay modificaciones concurrentes a estructuras compartidas (puede generar **condiciones de carrera**).
- Si el orden de los resultados es importante (`parallelStream()` **no garantiza orden**).

El problema del NullPointerException

El NullPointerException (NPE) ocurre cuando intentamos acceder a un método o atributo de un objeto que es null. Es una de las excepciones más comunes en Java y puede causar fallos inesperados en la ejecución del programa.

```
public class NPEExample {  
    public static void main(String[] args) {  
        String texto = null;  
        System.out.println(texto.length()); // ERROR: NullPointerException  
    }  
}
```

Veamos cual es la solución más óptima a partir de Java 8...

Optional <T>

Esta clase fue diseñada para evitar `NullPointerException` y mejorar la gestión de valores nulos de manera segura y declarativa.

En lugar de devolver null, `Optional<T>` permite representar valores presentes o ausentes, proporcionando métodos para manejar ambas situaciones sin necesidad de validaciones manuales con if.

```
public class OptionalBestPractice {  
    public static void main(String[] args) {  
        String texto = obtenerTexto().orElse("Valor por defecto");  
        System.out.println(texto);  
    }  
  
    public static Optional<String> obtenerTexto() {  
        return Optional.ofNullable(null); // Simula un valor nulo  
    }  
}
```

Principales métodos de Optional

- **.of(valor)**: Crea un Optional con un valor (no puede ser null).
- **.ofNullable(valor)**: Crea un Optional con un valor, o vacío si es null.
- **.isPresent()** : Devuelve true si el Optional tiene un valor.
- **.ifPresent(Consumer<T>)**: Ejecuta una acción si el valor está presente.
- **.orElse(valorDefecto)**: Devuelve el valor si está presente, o el valor por defecto si no lo está.

¿Cuándo usar Optional?

-  Para métodos que pueden devolver null, como **consultas a bases de datos o archivos**.
-  Para **evitar comprobaciones manuales** de null.
-  Para mejorar la **legibilidad y seguridad** del código.
-  No usar Optional en:
 - Parámetros de métodos (usar **Objects.requireNonNull()** en su lugar).
 - Atributos de una clase (usar valores predeterminados o validaciones).